

PAPER • OPEN ACCESS

A burst-dependent algorithm for neuromorphic on-chip learning of spiking neural networks

To cite this article: Michael Stuck *et al* 2025 *Neuromorph. Comput. Eng.* **5** 014010

View the [article online](#) for updates and enhancements.

You may also like

- [Bio-realistic and versatile artificial dendrites made of anti-ambipolar transistors](#)
Yifei Yang, Mingkun Xu, Tianhang Zhang et al.
- [Leveraging dendritic complexity for neuromorphic computing](#)
Suma G Cardwell, Mark Plagge, Luke Parker et al.
- [General and stable emulation of finite state machines with spiking neural networks](#)
Ziyang Sun, Zhong Zheng, Binying Zhang et al.



PAPER

OPEN ACCESS

RECEIVED
19 July 2024REVISED
2 January 2025ACCEPTED FOR PUBLICATION
12 February 2025PUBLISHED
20 February 2025

Original Content from
this work may be used
under the terms of the
[Creative Commons
Attribution 4.0 licence](#).

Any further distribution
of this work must
maintain attribution to
the author(s) and the title
of the work, journal
citation and DOI.



A burst-dependent algorithm for neuromorphic on-chip learning of spiking neural networks

Michael Stuck^{1,3} , Xingyun Wang^{2,3,4} and Richard Naud^{1,2,3,4,*} ¹ Department of Physics, University of Ottawa, 150 Louis Pasteur Pvt, Ottawa, ON, K1N 6N5, Canada² Department of Cellular and Molecular Medicine, University of Ottawa, 451 Smyth Rd, Ottawa, ON K1H 8M5, Canada³ Centre for Neural Dynamics and Artificial Intelligence, University of Ottawa, 150 Louis Pasteur Pvt, Ottawa, ON, K1N 6N5, Canada⁴ University of Ottawa Brain and Mind Research Institute University of Ottawa, 451 Smyth Rd, Ottawa, ON K1H 8M5, Canada

* Author to whom any correspondence should be addressed.

E-mail: rnaud@uottawa.ca**Keywords:** spiking neural networks, neuromorphic learning, supervised learning, synaptic plasticity, local learning algorithms, burst-dependent learning, credit assignment

Abstract

The field of neuromorphic engineering addresses the high energy demands of neural networks through brain-inspired hardware for efficient neural network computing. For on-chip learning with spiking neural networks, neuromorphic hardware requires a local learning algorithm able to solve complex tasks. Approaches based on burst-dependent plasticity have been proposed to address this requirement, but their ability to learn complex tasks has remained unproven. Specifically, previous burst-dependent learning was demonstrated on a spiking version of the ‘exclusive or’ problem (XOR) using a network of thousands of neurons. Here, we extend burst-dependent learning, termed ‘Burstprop’, to address more complex tasks with hundreds of neurons. We evaluate Burstprop on a rate-encoded spiking version of the MNIST dataset, achieving low test classification errors, comparable to those obtained using backpropagation through time on the same architecture. Going further, we develop another burst-dependent algorithm based on the communication of two types of error-encoding events for the communication of positive and negative errors. We find that this new algorithm performs better on the image classification benchmark. We also tested our algorithms under various types of feedback connectivity, establishing that the capabilities of fixed random feedback connectivity is preserved in spiking neural networks. Lastly, we tested the robustness of the algorithm to weight discretization. Together, these results suggest that spiking Burstprop can scale to more complex learning tasks and is therefore likely to be considered for self-supervised algorithms while maintaining efficiency, potentially providing a viable method for learning with neuromorphic hardware.

1. Introduction

The increasing energy demands of neural network-based artificial intelligence (AI) systems have created significant interest in developing more energy-efficient methods [1, 2]. Neuromorphic computing, which takes inspiration from the computational principles of the human brain, has emerged as a promising solution to reduce energy costs associated with AI systems. Two biological aspects, in particular, are thought to yield significant energy savings.

One is the event-based mode of communication [3–5]. Unlike traditional neural networks that rely on high-precision, continuous activations, SNNs operate using single-bit, binary activations, known as spikes. Running SNNs with low-precision parameters and high spatial and temporal sparsity levels contributes to this energy efficiency [1].

The other advantageous biological property is the co-localization of computing and memory which circumvents the von Neumann bottleneck [6, 7]. Current methods for training SNNs predominantly rely on conventional von Neumann computer architectures, with the learned synaptic weights transferred to spiking

neuromorphic hardware to enable energy-efficient inference. While using spiking neuromorphic hardware for inference has demonstrated energy savings in certain applications [8], further energy efficiency can be achieved by training SNNs directly on hardware. Direct hardware training reduces energy consumption and allows for continual learning, essential for systems that need to dynamically adapt to changing environments. Achieving this requires new training algorithms that are compatible with the unique constraints of neuromorphic hardware.

The constraints presented by neuromorphic hardware for learning have many similarities with those present in biological systems. As such, a learning algorithm for spiking neuromorphic hardware must include certain characteristics which are thought to be important for biologically plausible learning. The most essential of these biologically plausible features of learning is local, event-driven synaptic plasticity [9]. This means that synaptic weight changes must be triggered by either presynaptic, postsynaptic or global events and can depend only on the states of the presynaptic neuron, postsynaptic neurons, a global state and the synapse itself. A local alternative to backpropagation is necessary to do on-chip learning of SNNs in neuromorphic hardware.

One promising group of local learning methods for training SNNs are burst-dependent, ‘Burstprop’ or BurstCCN learning methods [10, 11]. These algorithms are grounded in the widely observed burst-dependence of synaptic plasticity [12–15] and synaptic dynamics which confers synapses with the ability to communicate spike-timing patterns differently to different targets. In hierarchical networks, Burstprop can approximate the backpropagation of error algorithm with single-phase local learning. It accomplishes this by an ensemble multiplexing [16] of error signals, which are transmitted backwards through the network and integrated by dendritic compartments to steer plasticity while the network processes feedforward information through somatic neuron compartments. Thus far, however, Burstprop methods have been limited to solving the ‘exclusive or’ (XOR) problem in SNNs [10, 11]. For Burstprop to be considered viable for neuromorphic learning applications, it must be capable of learning more difficult tasks. Similarly, we currently do not know if feedback alignment (FA), a technique used to perform credit assignment without an artificial tying of the connectivity, preserves its function in SNNs using local and online plasticity rules.

In this work, we improve and test current Burstprop methods, allowing it to scale to more difficult learning problems. We consider two improvements: (1) a regularizing feedback connection from the soma to the dendrite of every neuron and (2) communication of two types of bursts, allowing for the multiplexing of signed errors. We evaluate our algorithm using the MNIST digit classification and other related benchmarks (EMNIST and FMNIST) and show that it can successfully learn these tasks. We also investigate the impact of the choice of feedback connectivity and weight discretization. The results support that our algorithm is a viable method for supervised learning of SNNs for neuromorphic implementation, a first step for implementation of more elaborate algorithms such as large unsupervised, self-supervised or deep-reinforcement-learning models.

2. Background

2.1. Learning in SNNs

Multiple methods for training SNNs have been developed. We first distinguish methods relying on non-local plasticity rules from those using local ones. The local plasticity rules can be further separated into burst-dependent plasticity and spike-timing-dependent plasticity.

2.1.1. Non-local learning methods

Nonlocal learning methods for SNNs include ANN-SNN conversion [17–20] and backpropagation-through-time (BPTT) [21, 22]. With ANN-SNN conversion, a rate-based neural network is trained using the backpropagation algorithm which is then converted to a spiking neural network. In real applications, imperfect conversions will strongly alter performance, but this can be remedied with further fine-tuning [8]. With BPTT, backpropagation is performed on an SNN by using surrogate gradients to propagate the gradient through the spiking nonlinearity. To backpropagate gradients through time, the gradient must keep track of the network state at multiple time steps of past activity for each time step. This algorithm is therefore highly nonlocal (but local-only approximations have been derived [23]). These methods do not address the problem faced with training neuromorphic online, we will use these methods to provide a point of reference for the performance metrics.

2.1.2. Spike-time-dependent plasticity (STDP) learning methods

Spike-time-dependent plasticity (STDP) alters connectivity according to the relative timing of spikes in the presynaptic and postsynaptic neurons and is therefore fully local. One local learning algorithm, proposed by

Kappel *et al* [24] combines a STDP plasticity rule and winner-take-all dynamics and recurrent excitation to approximate hidden Markov model learning in SNNs. Relatedly, Diehl and Cook [25] combined STDP with winner-take-all dynamics to train a non-recurrent network. Reading out the resulting representation with a supervised plasticity rule, they showed that a single layer could achieve 95% accuracy on MNIST. As only the readout is supervised, this method is sometimes referred to as unsupervised. Another STDP approach combines a trace set by relative coincidence and a global reinforcement by reward-associated signals [26–28], but such three-factor learning rules tend to scale poorly on problems having a high number of dimensions. Also, how these STDP approaches can scale to multiple layers in a hierarchy remains unknown.

2.1.3. Burstprop learning methods

Another class of local learning algorithms depend on separate somatic and dendritic compartments to represent both feedback error signals and inference signals within the neuron, often through a rate-based representation [29–31, 67]. For spiking neural networks, burst-dependent algorithms exploit the target-dependence of short-term plasticity [32–34], dendrite-dependent bursting [35, 36] and burst-dependent plasticity [12–15] to perform credit assignment in a way that approximates backpropagation [10, 11, 37]. These burst-dependent learning methods, referred to as ‘Burstprop’ algorithms, can in principle allow for multilayer local learning in SNNs.

The original formulation [10] of the algorithm learns online but with signals coming in two phases. An initial phase in which synaptic plasticity was disabled was required to determine each neuron’s baseline burst probability which was then used in the second phase to calculate synaptic plasticity, violating temporal locality. A second formulation of the algorithm (called BurstCCN [11]) adapted the learning rule for single-phase learning by including a second feedback pathway that communicates spike signals such that a baseline burst probability can be preset, avoiding the need for two phases. Addressing complex problems with these algorithms has been almost entirely limited to rate-based networks. Spiking proof-of-principles were done on the XOR problem.

2.2. Weight-transport-free learning

Approaches approximating the backpropagation of error algorithm face a problem known as the weight-transport problem. To accurately approximate the gradient, the feedback pathways must have synaptic weights that are symmetric to feedforward weights (symmetric feedback). Such direct weight-transport involves nonlocal communication weight values between synapses and must be avoided for learning in spiking neuromorphic hardware. Many solutions have been proposed [10, 38–44], including imposing random but static feedback connectivity (FA [38, 39]) and learning rules that seek to transport the weights by exploiting correlations in the activity patterns [10, 41, 43].

2.2.1. FA methods

It has been shown that feedback weights need not be initialized as symmetrical to feedforward weights, nor to be kept tightly symmetrical throughout learning to effectively train networks using backpropagation [38]. As long as feedforward weights are initialized within 90° of feedback weights, feedforward weights trained with backpropagation tend to align with feedback weights, leading to accurate credit assignment and effective learning. Since feedforward synapses must align with feedback synapses, this method is referred to as FA. Building upon this work, direct FA (DFA) uses random feedback weights directly connecting network outputs to each layer and has also been shown to be effective for learning with backpropagation [39]. While these methods are effective for training shallow, rate-based neural networks, their efficacy tends to decrease with increasing network depth [45–47].

The question of whether these techniques also scale to spiking neural networks has been partially addressed in Samadi *et al* [40], where a spiking feedforward inference phase was combined with an analog learning phase that follows the backpropagation of error algorithm. Zenke and Ganguli [48] have similarly paired a spiking inference phased with analog, non-local, BPTT learning. Relatedly, Zhao *et al* [49] have studied SNNs receiving direct but analog gradient information. Xing *et al* [50] have used a signed spiking communication for the feedforward inference phase, but again paired with analog BPTT communication of gradients. In Payeur *et al* [10] a fully spike-based communication of both feedforward and learning signals, but the FA approach was only applied to the XOR problem. The present work focuses on MNIST in a fully spiking communication of both feedforward and learning signals.

2.2.2. Learned feedback

Several methods implement a learning rule on feedback synapses, allowing them to align with feedforward weights. With the Kolen-Pollack method [51], weight symmetry is achieved by assigning the same plasticity rule to feedforward and feedback synapses and including a weight decay term. While this is effective in

aligning weights, the necessity of weight decay may be a problem for some learning methods. Phaseless alignment learning aligns feedback weights to feedforward weights by learning feedback weights from neuronal noise [43] outside of the presentation of examples. Another algorithm proposed by Burbank [52] uses temporally opposed STDP learning rules on feedforward and feedback synapses to successfully train a spiking autoencoder network. Similarly, Payeur *et al* [10] provide a spike-based learning rule for learning the alignment of feedback synapses, which have not been tested on SNNs. These methods form a conceptual rationale for using implementing symmetry as a partially accurate bypass of the learned feedback method.

3. Methods

3.1. Neuron and synapse model

3.1.1. Burst coding

We consider a neural code in which neurons are thought to emit spikes in two different modes: either bursts or single spikes. Bursts are a set of action potentials emitted at a high frequency, typically 100 Hz [15]. There are at least 2 spikes per burst, but the number of spike per burst is not a requirement for its class. Single spikes are relatively isolated action potentials. Both bursts and single spikes are considered events. We note that in such a code, it has been shown that a small number of spikes per bursts and a low burst probability is maximizing information transmission [53], such that burst multiplexing is thought to be an energy efficient way to convey two signals at the same time.

In what follows, we will also be introducing the notion of two different types of bursts. We consider that a simple interpretation of such a code is to consider long bursts (e.g. bursts having more than 5 or 6 spikes) and short bursts (e.g. bursts having 4 spikes or less). While there is only limited evidence that different bursting modes (long and short) are regulated in the brain [15], we are not aware of any evidence to support the fact that such modes of firing would be regulated by different types of inputs to the same dendrite. As a consequence, we consider it is mainly avenue of interest for neuromorphic hardware implementations. Because of their relationship with the input signal, we will label these different types of bursts ‘positive’ and ‘negative’ bursts.

It is assumed based on experimental observation [10, 53] that inputs to the soma are driving the generation of any event, whether single spike or burst. The net inputs to the dendrite regulates whether this event is a burst or a spike in a probabilistic way, such that a population of neurons can multiplex two independent streams of information in its time-dependent event rate and time-dependent burst fraction. Evidence of this multiplexing has been observed experimentally [54].

3.1.2. Somatic compartment

Each neuron’s soma follows Leaky Integrate and Fire (LIF) dynamics [55]:

$$\tau_s \frac{dV_s(t)}{dt} = -(V_s(t) - E_L) + \frac{I_s(t)}{g_L} \quad (1)$$

where $V_s(t)$ represents the somatic membrane potential, τ_s the membrane time constant, E_L the resting potential, g_L the leak conductance, and $I_s(t)$ the input current. LIF models have been shown to be quantitatively accurate models of many different cell types especially when integrating spike-frequency adaptation and nonlinear dendritic compartments [56–60]. The two-compartment LIF model without adaptation was chosen so as to implement the simplest set of dynamics able to capture credit assignment by burst-dependent plasticity. In our experiments, the somatic membrane time constant was set to $\tau_s = 10$ ms. The neuron fires an event when $V_s(t)$ exceeds the threshold potential $V_{\text{threshold}}$, following which $V_s(t)$ resets to V_{reset} :

$$\text{if } V_s(t) \geq V_{\text{threshold}}, \text{ then } V_s(t) \rightarrow V_{\text{reset}}. \quad (2)$$

Rescaling to dimensionless variables, we define $\tilde{I}_s = I_s / g_L (V_{\text{threshold}} - E_L)$ and $\tilde{V}_s = (V_s - E_L) / (V_{\text{threshold}} - E_L)$, equation (1) becomes

$$\tau_s \frac{d\tilde{V}_s(t)}{dt} = -\tilde{V}_s(t) + \tilde{I}_s(t). \quad (3)$$

Every time the somatic voltage crosses the threshold, an event is emitted. As mentioned in the previous section, this event can be either a single spike or a burst, as determined by the dendritic state (see section 3.1.4).

3.1.3. Eligibility trace

We use eligibility traces, taken as the exponential moving average of the event train $\bar{E}_j(t)$, to indicate the eligibility of individual synapses connecting from a lower-order neuron j to some other higher-order neuron:

$$\tau_E \frac{d\bar{E}_j(t)}{dt} = -\bar{E}_j(t) + S_j(t). \quad (4)$$

Here the event train $S_i(t) = \sum_k \delta(t - t_k^{(i)})$, where $t_k^{(i)}$ is the k th event time from neuron i . The eligibility trace time constant was to $\tau_E = 10$ ms.

3.1.4. Dendritic compartment and burst generation

Dendrites come with different properties to the neuron's soma [61]. Here the dendritic compartment refers to an apical-like dendrite with a segregated membrane potential that can nonlinearly integrate feedback information to modulate neuron burst generation [35, 59, 62, 63]. It follows leaky integrator dynamics, using dimensionless inputs and membrane potential:

$$\tau_d \frac{d\tilde{V}_d(t)}{dt} = -\tilde{V}_d(t) + \tilde{I}_d(t). \quad (5)$$

where $\tilde{V}_d(t)$ represents the re-scaled dendritic membrane potential, τ_d is the dendritic membrane time constant, and $\tilde{I}_d$ is the re-scaled net dendritic input. We set the dendritic membrane time constant to $\tau_d = 10$ ms.

There is no threshold in the dendritic compartment, instead, every time the soma emits an event, the state of the dendrite is used to determine if this event is a burst or a single spike. We consider two bursting models.

In the unsigned bursting model, the probability that an event is made into a burst is proportional to the dendritic potential $\tilde{V}_d$ [64], we use a sigmoidal relationship $P_B(\tilde{V}_d(t)|\text{event at } t) = \frac{1}{1+e^{-\tilde{V}_d(t)}}$.

In the signed bursting model, two types of bursts can be emitted (e.g. long bursts and short bursts, see Burst Coding section). Either type of burst can only be generated upon the firing of an event from the soma. Given a somatic event, a burst is generated according to the symmetric function $P_B(\tilde{V}_d(t)|\text{event at } t) = \tanh(\text{abs}(\tilde{V}_d))$. If a burst is said to be generated, the sign of $\tilde{V}_d(t)$ is used to determine its type. If $\tilde{V}_d < 0$, a short burst is generated. If on the other hand $\tilde{V}_d > 0$, a long burst is generated. Because they represent the sign of the dendritic input, we refer to short and long bursts as negative and positive bursts, respectively. In this model, burst generation triggers a reset of the dendritic potential to the baseline, comparable to the somatic membrane potential reset upon a neuron spiking. This adds some refractoriness allowing the algorithm to better approximate backpropagation.

3.2. Network architecture

The SNNs consist of an input layer of 784 units, corresponding to the number of pixels in each sample image in the MNIST-like datasets. There are 1, 2, 3 or 4 hidden layers consisting of 100, 200, 400 or 800 hidden neurons as specified in the figures and associated text. The output layer of 10 neurons corresponds to the 10 classes of the MNIST dataset. Except for input units, each unit follows the two-compartment model described above. The network inputs (see section 3.8), as well as any connection between hidden layers or from the hidden layer to the output layer follow the fully connected configuration. Weights are allowed to be positive or negative. The label corresponding to the network's class is used in calculating the network's output error and teaching signal (see section 3.4.1).

3.3. Synaptic communication and multiplexing

In our fully connected hierarchical SNNs, the credit-carrying information is communicated by multiplexing feedforward signals using events and feedback error signals using bursts. For the feedforward signals, we assume that synapses operate instantaneous changes in postsynaptic membrane potential upon every event. The input $I_s^{(i)}(t)$ to the soma of neuron i is defined by the equation:

$$I_s^{(i)}(t) = \sum_j w_{ij} S_j(t). \quad (6)$$

In this equation, w_{ij} is the feedforward synaptic weight from lower-order neuron j to higher-order neuron i .

3.3.1. Unsigned burst communication

For the feedback signals, we again assume that synapses operate instantaneous changes in postsynaptic membrane potential, but upon every burst. We further enforce that these feedback connections target the

apical dendrite and therefore only affect the dendritic input $I_{\text{FB}}^{(i)}(t)$ of a given neuron i . In the unsigned burst model, we have

$$I_{\text{FB}}^{(i)}(t) = \sum_j b_{ij} \hat{B}_j(t). \quad (7)$$

where b_{ij} are the feedback synaptic weight from higher-order neuron j to lower-order neuron i and $\hat{B}_j(t)$ is the burst train of neuron j . Other, locally generated currents will combine with this feedback signal, see section 3.4.2.

3.3.2. Signed burst communication

In the signed burst model, we assume the type of burst can dictate the sign of the post-synaptic effect:

$$I_{\text{FB}}^{(i)}(t) = \sum_j b_{ij} \left(\hat{B}_j^+(t) - \hat{B}_j^-(t) \right). \quad (8)$$

We note that since the same feedback weight b_{ij} operates on two distinct outputs of a neuron, this model requires weight transport.

3.3.3. Multiplexing

In both signed and unsigned bursting models, when a neuron fires an event, the probability of that neuron sending a positive or negative error signal backward in the network is modulated by the dendritic potential of the neuron, V_d , as demonstrated in figure 2. Similarly, the event rate is proportional to the integrated somatic input (proportional to V_s if there were no reset). Thus the burst probability and the event rate represent two different signals simultaneously. Since each neuron integrates error signals from all higher-order neurons in its dendritic compartment and through the burst generation mechanism, the burst probability signal represents the signal that started with the input onto the apical dendrites at the top of the hierarchy and then backpropagates through it. This allows for the simultaneous online communication of credit-carrying error information from higher to lower-order neurons and feedforward, inference signals from lower to higher-order neurons.

3.4. Teaching signals and target rates

For dendritic inputs to be conceived as providing error signals, we inject a teaching signal at the top of the hierarchy (section 3.4.1) and a unit-specific regularisation signal (section 3.4.2), which come together in some layers but not others (section 3.4.3).

3.4.1. Teaching signals

Teaching signals are used for supervision, applying them to the dendritic compartments of output neurons. These signals are derived from the exponential moving average of the neuron's firing rate, $\bar{E}(t)$, using the same time constant as for the eligibility trace (see section 3.1.3). The teaching signal received by the dendrites at the top of the hierarchy, $T_i(t)$ is calculated as the derivative of the local squared error, $\epsilon_T(\bar{E}_i(t), \hat{E}_i)$, of the neuron's instantaneous firing rate, $\bar{E}_i(t)$ with respect to the neuron's target rate, $\hat{E}_i$. For the neuron corresponding to the correct output class, a high target rate of $\hat{E}_i = 200$ Hz is set, and for all other neurons, a low rate of $\hat{E}_i = 20$ Hz is used. With this target, the teaching signal becomes

$$T_i(t) = \gamma_T \frac{\partial \epsilon_T}{\partial \bar{E}(t)} = -2\gamma_T (\hat{E}_i(t) - \bar{E}_i(t)), \quad (9)$$

where ϵ_T corresponds to a square error loss. Here γ_T controls the strength of the teaching signal. This differential in target rates will backpropagate and control synaptic plasticity through the network, ensuring the correct output neuron becomes more active while other output neurons are suppressed. We found that a teaching signal strength of $\gamma_T = 10^{-3}$ produced a strong enough teaching signal to steer plasticity while avoiding error signal saturation. Unlike in previous implementation of burstprop [10] we did not force the teaching signal to come after a delay, but the 100 ms was long enough to ensure that inference signal had ample time to reach the output layer and the associated gradient had ample time to reach the first hidden layer.

3.4.2. Hidden target rates

Similar to the teaching signals applied to output neurons, hidden neurons receive a regularization signal that we model as a self-connection, such that $I_{\text{self}}^{(i)}(t) = R_i(t)$ for all neurons except input neurons and output neurons. We use a uniformly low target rates $\hat{E} = 20 \text{ Hz}$:

$$R_i(t) = \gamma_R \frac{\partial \epsilon_R}{\partial \bar{E}(t)} = -2\gamma_R (\hat{E} - \bar{E}_i(t)). \quad (10)$$

for ϵ_R a squared error loss. This $R_i(t)$ affects the apical dendrite of the same layer as the smoothed spike train $\bar{E}_i(t)$, hence the self connection. The parameter γ_R acts as a weight, but is conceived as a non-plastic hyper parameter. We set $\gamma_R = 10^{-5} \ll \gamma_T$ such that the regularization does not suppress the error information due to the teaching signal. This approach regulates the firing rates of hidden neurons, thereby promoting sparsity and stability in the network's learning process.

3.4.3. Dendritic inputs

Putting this together, we have that output units receive a dendritic input equal to the teaching signal and the regularizing signal

$$I_d^{(i)} = T_i(t) \quad (11)$$

where i is the label of one of the output units. The hidden units receive a combination of the burst-coded feedback signals and the regularization signal:

$$I_d^{(i)} = I_{\text{FB}}^{(i)}(t) + R_i(t) \quad (12)$$

where i is the label of one of the hidden units. Input units are modeled as having no dendrites.

3.5. Plasticity rule

Our algorithm utilizes a local spike-based learning rule to approximate backpropagation-based gradient descent. The synaptic weight changes are calculated as

$$\frac{dw_{ij}}{dt} = -\eta \tilde{E}_j(t) (\hat{B}_i^+(t) - \hat{B}_i^-(t)) \quad (13)$$

for the signed bursting model, and

$$\frac{dw_{ij}}{dt} = -\eta \tilde{E}_j(t) (\hat{B}_i(t) - \hat{B}_p S_i(t)) \quad (14)$$

for the unsigned bursting model. Here, B_p is the baseline burst probability. The parameter η represents the learning rate and was set to $\eta = 10^{-4}$ to achieve stable learning. We note that in the symmetric feedback scenario, the update of the feedforward weights w_{ij} also triggers an update in the feedback weight onto dendrites b_{ji} .

For the signed bursting model, synaptic weight changes, occur upon postsynaptic burst generation, with the sign of the weight change determined by the type of burst. For the unsigned bursting model negative weight changes occur upon postsynaptic spikes and positive weight changes occur upon postsynaptic bursts. In both models, the magnitude of plasticity is proportional to the presynaptic neuron's eligibility trace and the postsynaptic event rate. Plasticity, then, depends on three factors: the activity of the presynaptic neuron (via the eligibility trace), the postsynaptic neuron's activity (triggering plasticity), and the integrated upstream error (via the burst fraction or burst type fraction). These three factors are essential to emulate the backpropagation of error algorithm, where weight adjustments of a given unit depend on three factors: the incoming activity, the derivative of the unit's activity with respect to the sum of its inputs, and the integrated downstream error.

It is worth noting the distinct behaviour of signed and unsigned conditions in the absence of any postsynaptic error ($\tilde{V}_d(t) = 0$). For the unsigned burst model, this condition results in a burst probability of 0.5. Because $\tilde{V}_d = 0$ corresponds to the absence of dendritic input, we call the resulting output burst probability the baseline burst probability. In the associated plasticity rule, the parameter, $B_p = 0.5$, is chosen to match this baseline burst probability, such that there is a balance between positive and negative weight changes. We note that although this ensures that $\Delta w = 0$ on average when $\tilde{V}_d(t) = 0$, the fluctuations around this can cause drifts in the learned state over time. For the signed burst model, plasticity is more stable because the probability of burst generation is 0 when $\tilde{V}_d(t) = 0$ so no plasticity can occur.

3.6. Low-resolution weights

To implement low-resolution synaptic weights, $N^2 - 1$ evenly spaced possible weight values were determined ranging from -1 and 1, where N is the weight resolution in bits. The magnitude of the interval between possible weight values is referred to as ϵ . This low resolution was imposed on initial weights and required an adaptation of the plasticity rule.

Initial weights (see section 3.7) were rounded to the nearest possible weight value. During learning, an additional plasticity accumulator variable is added for each synapse. Plasticity is calculated according to the learning rule and added to the accumulator variable. Once a synaptic accumulator reaches $\pm 0.5\epsilon$, the synaptic weight potentiates or depresses by ϵ and the accumulator variable weight resets. In this way, the network can learn with low-resolution weights while still having an effective learning rate of less than ϵ .

3.7. Weight initialization

Feedforward synaptic weights were initialized from a normal distribution with standard deviation proportional to the square root of the number of presynaptic neurons to ensure stable learning [65]. It was found that to ensure error signals do not saturate, feedback weights had to be scaled by the number of higher-order neurons divided by the number of lower-order neurons in the layers connected by these weights. For the symmetric feedback implementation, feedback weights are aligned to feedforward weights. For FA and DFA, feedback weights were initialized from a normal distribution with the standard deviation scaled according to the number of higher-order neurons divided by the number of lower-order neurons while ensuring initial feedback weight matrices are within 90° of feedforward weights, as is done in [38].

3.8. Input encoding

In our study, we encoded the MNIST Digit Classification Dataset images for a spiking neural network. The dataset consists of 60 000 training and 10 000 test grayscale images, each 28×28 pixels [66]. The images were normalized so that the average pixel intensity of each image was constant. Pixel intensities, ranging from 0 to 1, were linearly mapped to firing rates between 20 and 200 Hz following a Poisson process. Accordingly, each pixel in each image was converted into an input unit spike train of a duration of 100 ms.

3.9. Simulation methods

We simulated the SNNs using a CPU with Python. Neuron dynamics were simulated using the forward Euler method with a 1 ms time step. For each time step, the network was updated from the input layer to the output layer. Feedforward inputs to a layer were taken from the lower-order outputs at the same time step and feedback inputs to a layer were taken from the higher-order outputs at the previous time step.

3.9.1. Training

During training, samples from the MNIST dataset are presented to the network sequentially. Teaching signals are applied continuously, and plasticity is engaged throughout the 100 ms. Neuronal state variables ($\tilde{V}_d$ and $\tilde{V}_s$), eligibility traces ($\tilde{E}$) and rate estimates ($\tilde{E}$) are reset to zero at the end of each sample presentation.

3.9.2. Testing

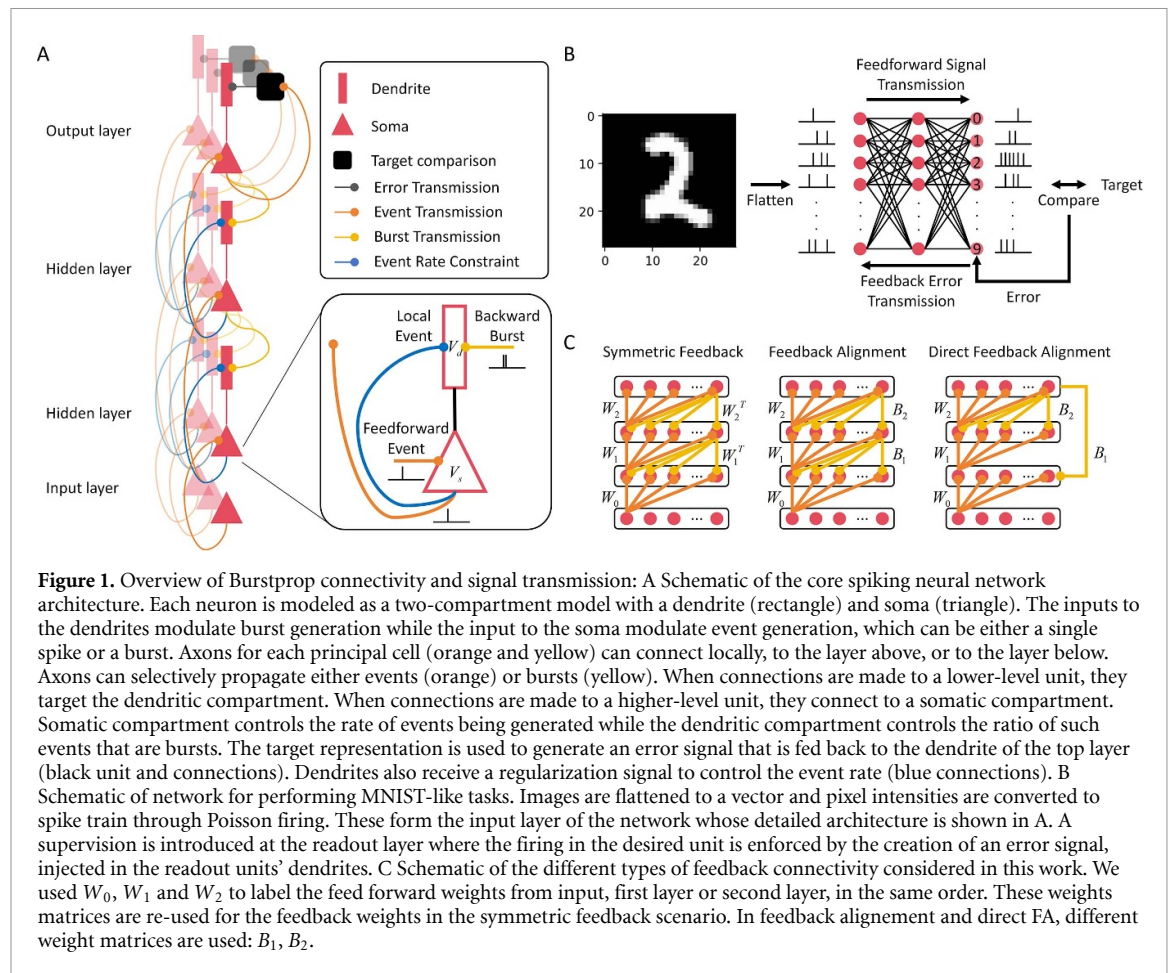
During testing, test samples are presented the same way as in the training phase, but there is no inputs to the dendrites and plasticity is off. The most active output neuron over the sample duration decides the network's predicted class.

3.9.3. Test in different conditions of initial sparsity

Different sparsity of connections were studied by initializing non-zero weights with different probability from 0.0 (no sparsity) to 0.8 (high sparsity). More specifically, we used the dropout function in Pytorch to set initialized weights to be zero with probability p layer by layer both for forward weights and backward weights. We note that the probability p does not correspond to the sparsity of the trained network as connections were allowed to change their weights from zero activity. Simulations where sparsity was changed were different set of simulation carried not only on the MNIST dataset but also on EMNIST' and FashionMNIST. In these simulations, the number of training samples and test samples are 10 000 and 1000.

3.10. BPTT

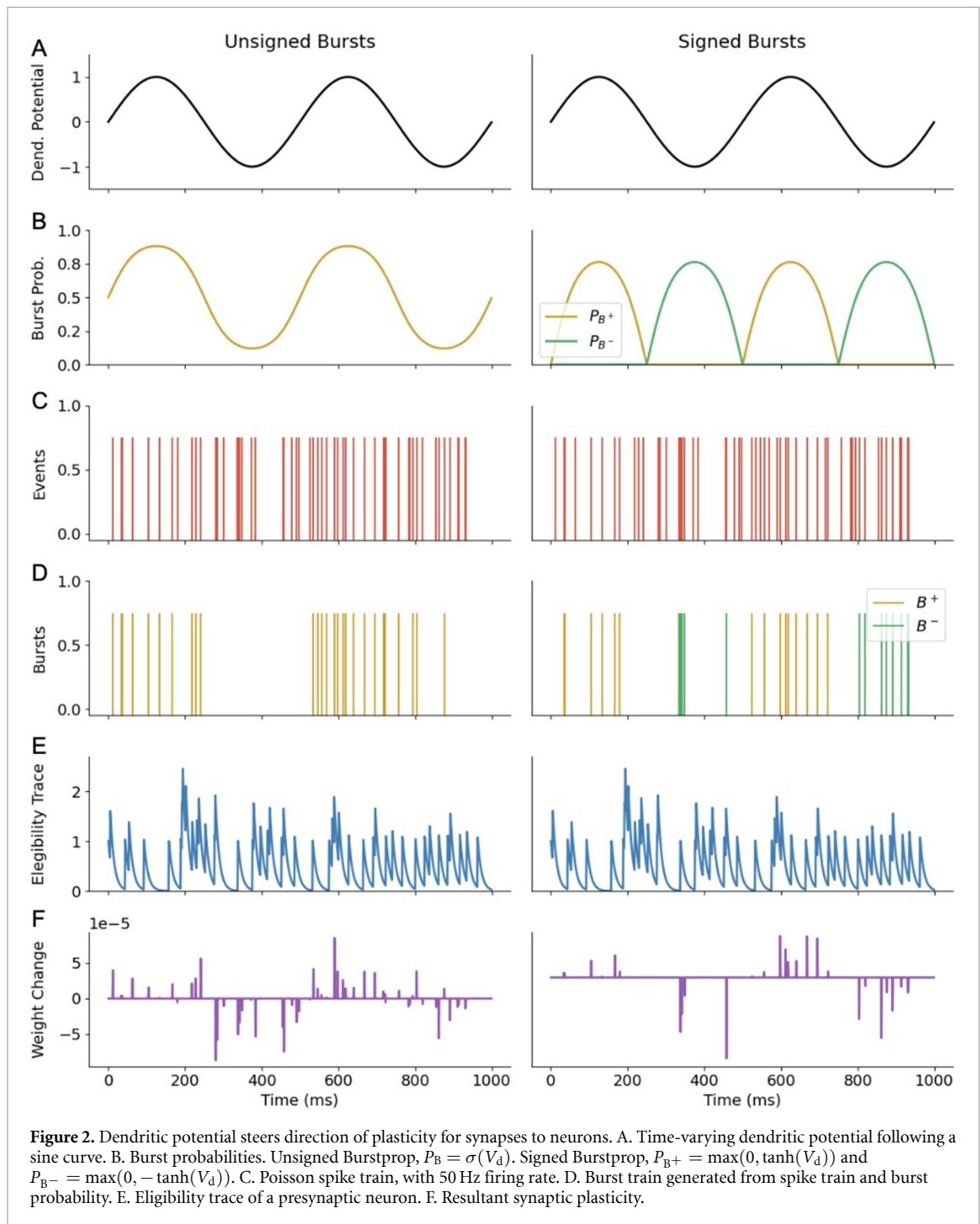
We compare our results with a BPTT method using surrogate gradients [21]. We use the sigmoid function to approximate the spike discontinuity in the backward pass. The neurons follow the same dynamics, with the same parameters as the somatic neuron compartments previously described. We trained the network on the same input encoding, with each sample presented also for 100 ms and used a learning rate of $\eta = 2 \times 10^{-4}$ to balance stability and speed of learning.



4. Results

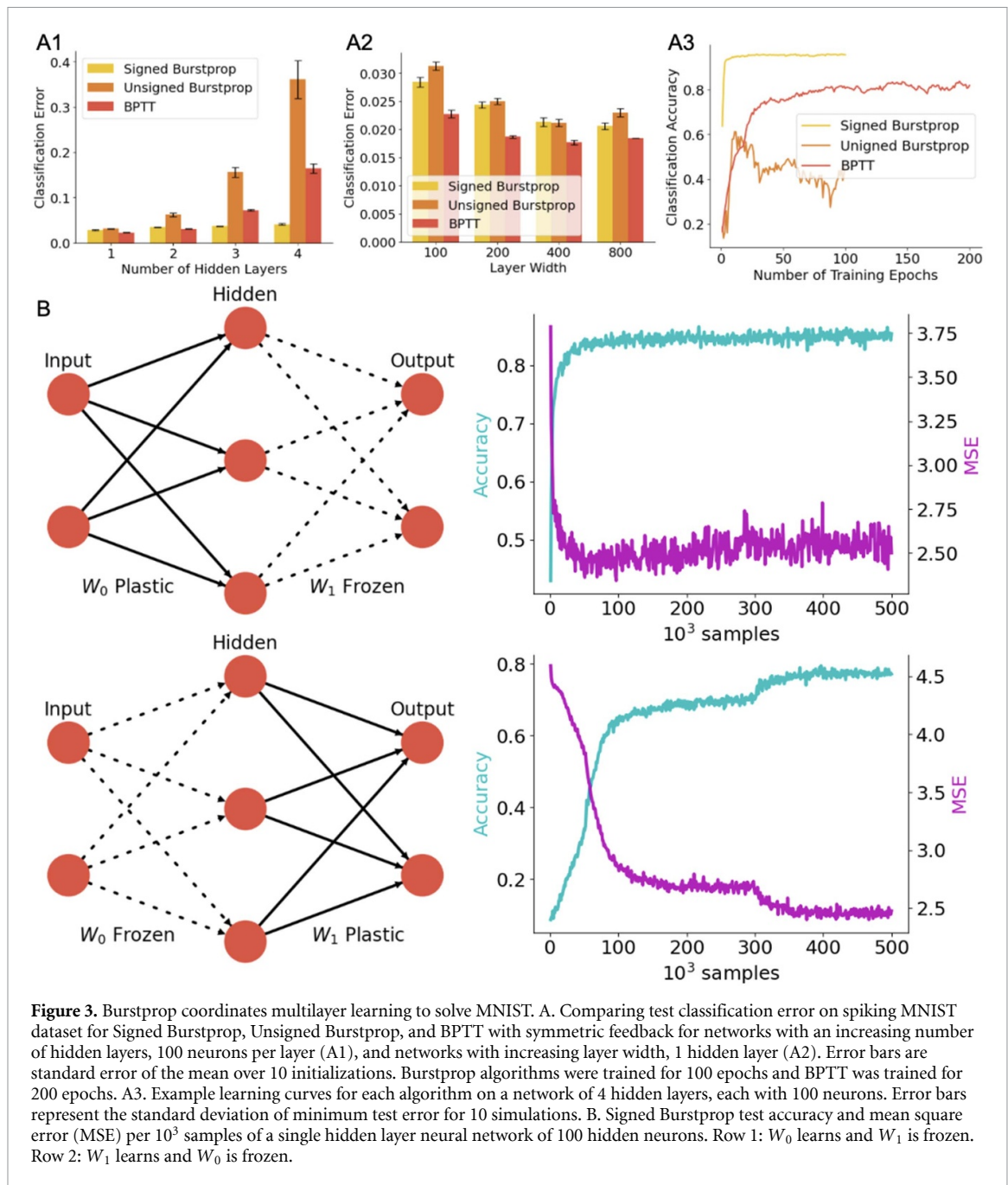
We trained SNNs with Burstprop [10] on a rate-coded version of the MNIST dataset. The idea is to use burst multiplexing to represent simultaneously the inference signal and the gradient signals, allowing for online learning with local learning rules. Burst multiplexing considers the neural code as being made with two types of events: either single spikes or bursts of high frequency action potentials. In a common implementation of this neural code, inputs to the dendrites controls the fraction of events that are bursts while the inputs to the soma control the rate of event emission. The present implementation uses two-compartment neuron models with LIF soma and dendrite-dependent bursting. The multiplexed signals can be sent to different neurons via frequency-dependent communication: bursts are communicated to one target while events (both bursts and singles spikes) are communicated to another target. Burst multiplexing does not use time to alternate between two different inputs like in the time-division multiplexing common in modern electronics. The code is closer to frequency-division multiplexing as the immediate interval is used to separate bursts from single spikes. Yet, burst multiplexing remains distinct from frequency-division multiplexing as the demultiplexed signals re-use bursts for the representation of the two signals. We will use the dendrite-encoded burst fraction to represent the gradient and the soma-encoded event rate to represent the inference signals.

To achieve this, we wire the network so as to keep gradient and inference signals separated. Specifically, all events are sent to somata up the hierarchy (figure 1(A), orange links) while only bursts are sent down the hierarchy and these axons only target dendrites (figure 1(A), yellow links), thus multiplexing feedforward signals with feedback signals. To ensure that the feedback signals are carrying credit information, we ensure the dendrites at the top of the hierarchy receive the classification error (figure 1(B)). The mathematical formulation of burst generation and plasticity are then chosen so as to approximate the backpropagation of error algorithm for rate-based representations (see supplementary information of [10]). We note that the dendrites could in principle receive other signals than a supervised error signal. For instance, prediction errors signals in the dendrites would allow for the implementation of self-supervised algorithms [67]. We also note that the role of the precise structure of feedback connections has long been recognized as important and will be studied in section 4.2 (figure 1(C)).



While Burstprop uses increases and decreases of burst probability to communicate positive and negative signs of the error signals, here we also considered an alternative model where the different sign of the error signal triggers different types of bursts. Physiologically, this ‘Signed Burst’ model could correspond to bursts of different durations engaging different short-term plasticity [34] and long-term plasticity. By construction, the signed burst model is more stable and less subject to drifts in plasticity than the original Burstprop model.

In figure 2, we illustrate the two different types of bursting in with the imaginary case of a sinusoidal error signal. Errors are practically never sinusoidal but they can switch from positive to negative in the course of inference and learning samples. Whereas, in the unsigned burst model, the burst probability oscillates around a fixed mean and it is by crossing this mean level that the neurons are able to represent positive and negative errors (figure 2(B), left), in the signed burst model we have two burst probabilities, one for each type of burst, and the mean level is inconsequential for telling the sign of the error.



4.1. Deep learning on MNIST dataset

To test the performance of our SNNs, we have trained both signed and unsigned versions of networks using symmetric feedback, varying width and depth on the rate encoded MNIST data (figure 3). We found by inspecting the resulting test-set errors that both signed and unsigned models were able to effectively train spiking neural networks with one layer with the signed model achieving $97.2 \pm 0.1\%$ and $96.8 \pm 0.1\%$ test classification accuracy respectively on a network with a single hidden layer of 100 neurons. For deeper networks, the signed model outperformed the unsigned one (figure 3(A1)), with the signed model achieving $95.9 \pm 0.1\%$ accuracy compared to $64 \pm 1\%$ accuracy for the unsigned model on a 4 hidden layer network. We did not observe an increase but rather a decrease in performance with the number of hidden layers. While there was no performance gain with depth, increased width was associated with an increase in performance (figure 3(A2)). Our algorithms could thus successfully train SNNs to perform MNIST with a performance that increases with layer width. Furthermore, we conclude that the signed burst model helps the algorithm to remain successful in deep networks.

To contextualize the learning ability, we compared the test-set prediction accuracy of our local learning algorithm to a state-of-the-art, non-local, BPTT-based learning method [21]. While burstprop aims to approximate the back-propagation of error algorithm, we could not directly compare with this algorithm as

no good equivalent for SNN is commonly used. We found that in shallow networks, BPTT training applied on the same SNN architecture provided slightly better performance, with $97.7 \pm 0.1\%$ accuracy on a network with a single hidden layer of 100 neurons. This improvement was maintained across the different network widths (figure 3(A2)). Networks trained with BPTT did not show a decrease in performance with network depth ($84 \pm 1\%$ accuracy on a 4 hidden layer network), an observation that echoes previous tests [65]. Because the choice of hyperparameters is likely to influence the comparison of these three algorithms, we conservatively conclude that SNNs with local burst-dependent plasticity rules can achieve good performance on MNIST, even with a small number of neurons and with multiple layers.

That the network performance does not increase with network depth may suggest that the algorithm is only training the output weights. To investigate this, and to determine whether the algorithm is effectively assigning credit to earlier layers, we examine how freezing either the output weights or the input weights affects learning in a single hidden layer network. If learning were limited to the output layer, freezing plasticity in this layer should result in the network being unable to learn. Instead, what we see is that the network remains able to learn effectively when the output layer is frozen (figure 3(B)). To take another approach, forcing the plasticity to only occur in the output weights should preserve the performance if credit signals were not reaching the lower-order connection. We found that freezing the output weights has a smaller effect on learning capabilities than freezing the input weights. Freezing the input weights made learning much slower (figure 3). Together, our burst-dependent SNN can successfully propagate credit-carrying information to neurons and accurately steer plasticity beyond the output layer. This suggests that our algorithm is effectively approximating backpropagation in the way it assigns errors.

4.2. FA

While we have shown that Burstprop can successfully train an SNN on MNIST, we did this by enforcing symmetry between feedforward and feedback weights. To allow for neuromorphic learning, and to address biological plausibility, our algorithm must remain effective with weight-transport-free learning. We thus tested the performance of Burstprop on our spiking MNIST dataset using FA and DFA, two algorithms that avoid the weight-transport problem by using random but fixed feedback weights (see Methods and figure 1(C)). If Burstprop can learn from the data while using random but static feedback weights, this would suggest that weight transport is not necessary for learning in SNNs. We found that both signed (figure 4(B1)) and unsigned (figure 4(B2)) versions of the algorithm can achieve good performance, achieving $95.6 \pm 0.1\%$ accuracy and $85 \pm 1\%$ accuracy respectively on a 4 hidden layer network with FA. The decrease in performance with the number of layers was lessened by switching to FA for the unsigned burst model but unaltered for the signed burst model.

Next, we again investigated how freezing either the first or second layer impacts learning. In principle, learning the set of weights should not be possible if the higher-order layers have not had a chance to learn and align with the feedback [38]. Accordingly, if we freeze the output weights, learning should be strongly affected. This is indeed what we see (figure 4(B)), whereby freezing the output weights in an FA scenario completely blocked learning. Thus using random but fixed feedback weights can implement weight-transport-free learning in SNNs using local learning rules.

4.3. Testing EMNIST and FMNIST

MNIST is recognized as an exceedingly easy task for in machine learning as a linear classifier, especially paired with a sufficiently expressive input basis, already achieves over 90% accuracy [66]. In addition to the weight-freezing manipulation, we also tested the learning algorithm on other, more difficult but closely related tasks. As our algorithm is meant to approximate backpropagation, we do not expect that temporal tasks would be a good choice as these tasks typically require the use of biologically challenging algorithms such as backprop-through time [21]. Because of the computational load of simulating SNNs on traditional hardware, we restricted our tests to EMNIST and FMNIST. Averaging over 5 different initialization with different initial degree of connection probability, we found that while MNIST achieved an accuracy of $91.0 \pm 2.2\%$, the network achieved $91.1 \pm 1.5\%$ on EMNIST and $82.6 \pm 0.5\%$ on FMNIST.

We found that the performance was relatively better for EMNIST [68] and FMNIST [69]. The accuracy reached for EMNIST was 91.8 % and 81.2 % for FMNIST, that is higher or equal accuracies than for MNIST. This is to be compared with linear classifiers which achieve over 90 % on MNIST [66] and 56 % on EMNIST [68] and 83 % on FMNIST [69]. The performance of traditional neural networks also tends to be lower on EMNIST and FMNIST than on MNIST [70, 71].

4.4. Sparse connectivity

Because many hyper-parameters can have a profound influence on performance, we wanted to test our algorithm further. We chose to focus on the role of the sparsity of the initial connectivity because 1) sparsity

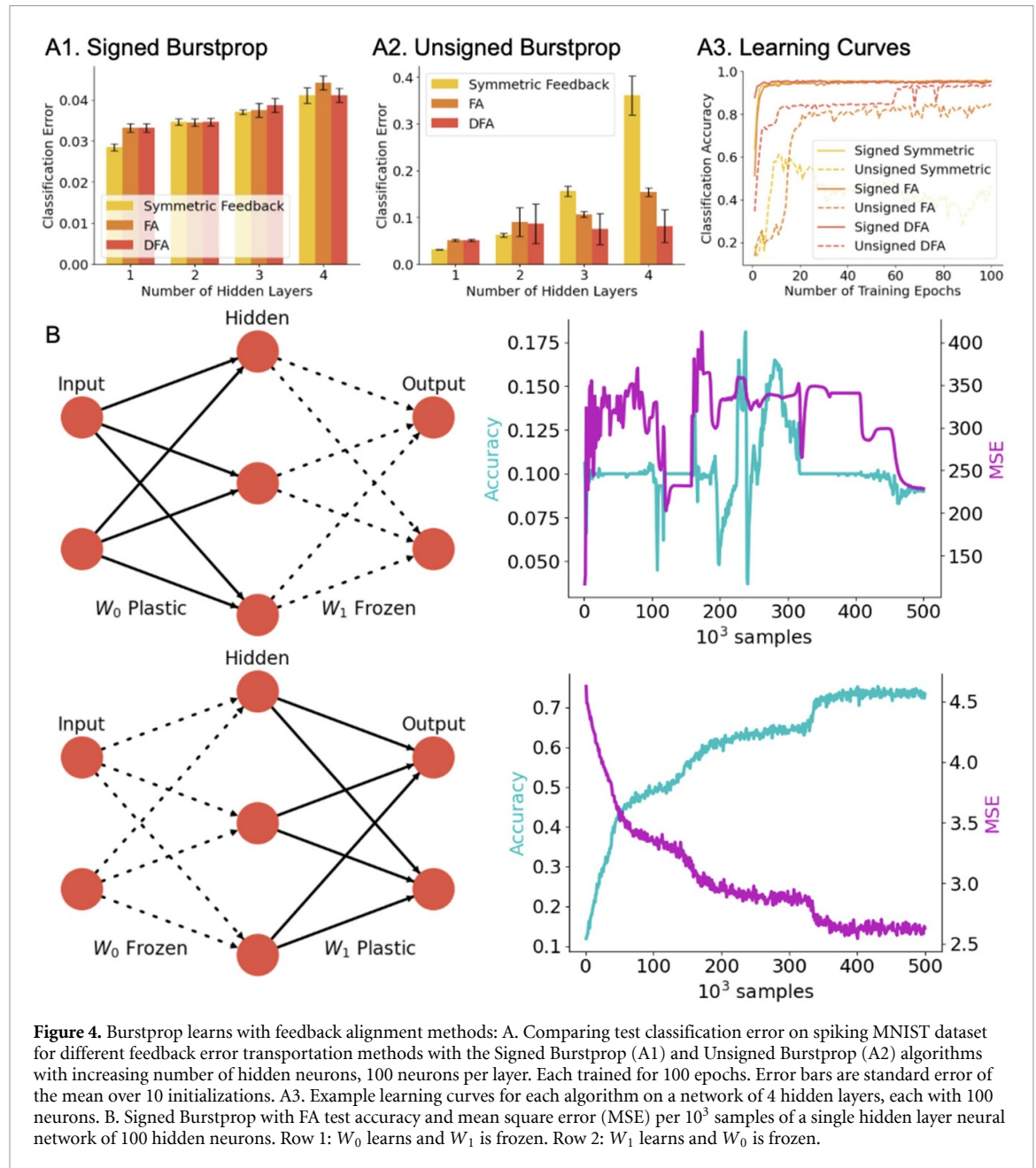
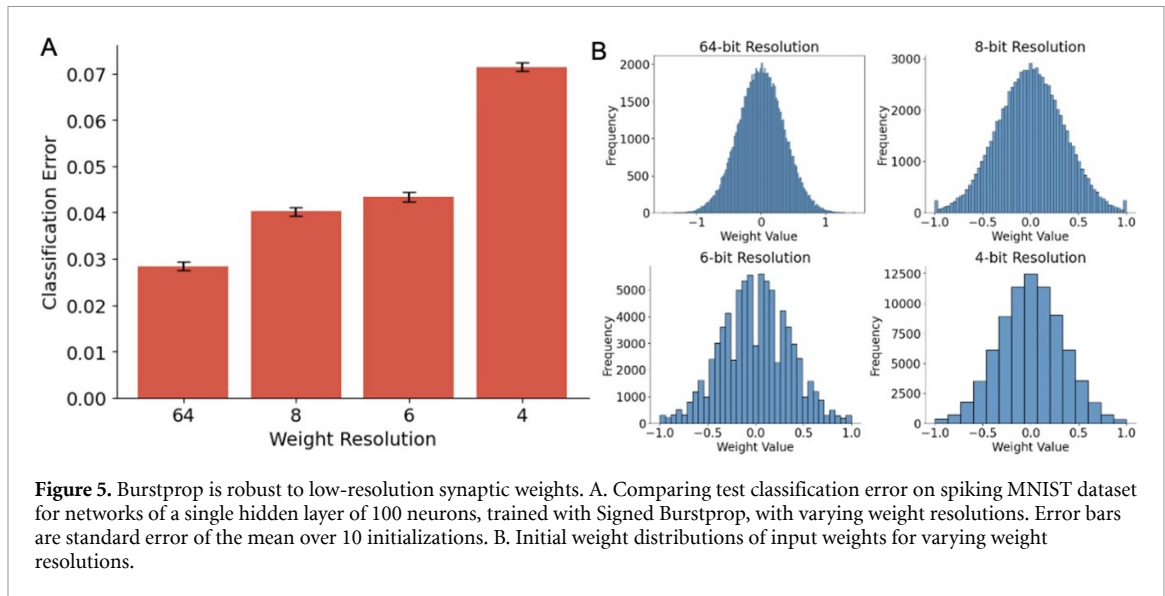


Figure 4. Burstprop learns with feedback alignment methods: A. Comparing test classification error on spiking MNIST dataset for different feedback error transportation methods with the Signed Burstprop (A1) and Unsigned Burstprop (A2) algorithms with increasing number of hidden neurons, 100 neurons per layer. Each trained for 100 epochs. Error bars are standard error of the mean over 10 initializations. A3. Example learning curves for each algorithm on a network of 4 hidden layers, each with 100 neurons. B. Signed Burstprop with FA test accuracy and mean square error (MSE) per 10^3 samples of a single hidden layer neural network of 100 hidden neurons. Row 1: W_0 learns and W_1 is frozen. Row 2: W_1 learns and W_0 is frozen.

Table 1. Test performance under different conditions of sparsity p —single simulation per condition. A high value of p means the network was initialized with many weights put to zero.

	MNIST		EMNIST		FMNIST	
	accuracy	error	accuracy	error	accuracy	error
$p = 0.0$	0.945	1.771	0.941	1.803	0.836	2.193
$p = 0.2$	0.941	1.877	0.933	1.858	0.838	2.255
$p = 0.4$	0.929	1.912	0.917	1.773	0.83	2.332
$p = 0.6$	0.926	1.758	0.847	2.119	0.813	2.335
$p = 0.8$	0.811	2.192	0.918	1.923	0.812	2.179

can reduce catastrophic forgetting [72] and 2) cortical neural networks are exceedingly sparse [73]. We simulated learning on all three benchmark datasets while varying the sparsity of the initial connectivity from 0 to 0.8 (table 1). We found that sparser initializations of the networks had a non significant tendency to degrade the test accuracy with increasing sparsity (Pearson $r = -0.43$, $P = 0.11$).



4.5. Low-resolution synaptic weights

An important consideration for implementing SNNs in neuromorphic hardware is the number of distinct values that synaptic weights can take [74]. Spiking neuromorphic hardware may only be able to support small bit resolutions for their weights, in the range of 4 to 8 bits [74]. Here we investigate the impact of decreased synaptic weights on learning performance. Figure 5 shows that learning performance decreases with weight resolution. Still, even with only 4-bit resolution, learning is achieved with over 90% classification accuracy. This result indicates that Burstprop is robust to low bit resolution in the synaptic weights.

5. Discussion

We have implemented a spiking version of the Burstprop algorithm that uses local learning rules and marked point processes for differential communication to different post-synaptic targets. Our algorithm improved on the previous efforts solving XOR with over a few thousand neurons to solving more complex tasks with a few hundred neurons. We found that our SNN using a local, burst-dependent learning rule could achieve a performance similar to that of BPTT. By experimenting with freezing weight layers during learning, we conclude that the algorithm can accurately perform credit assignment beyond the output layer. Training the algorithm with FA and DFA also showed that Burstprop can successfully perform weight-transport-free learning. The results further show the algorithm is robust to low-bit resolution synaptic weights. These results indicate that burst-dependent communication and plasticity can form an effective, biologically inspired, local learning method for SNNs that may be useful for neuromorphic learning.

We have shown that FA methods can be used effectively with SNNs using Burstprop to solve weight transport between feedforward and feedback weights. Our signed burst algorithm, however, requires some symmetry between two sets of feedback weights. Biological neurons would need two different synapses to communicate different firing patterns. These symmetric pathways are required to transmit positive and negative error signals. In our experiments, we have been enforcing this symmetry through weight transport, however, many methods could be implemented to align these weights. Further work must be done to find a suitable solution to address this problem. This problem is addressed in previous work [11] by adding a learning rule to feedback synapses, ensuring they converge.

Another weakness of our algorithm is that its fitting capabilities were not improved by increasing network depth. This is common for spiking neural networks [49, 65] and it is unclear whether this is related to the input encoding or could be a fundamental limitation of the algorithm. While neural networks often rely on deep structures to fit complex functions, even a single-layer spiking neural network is a universal function approximator so width could be used to compensate for depth in some applications. A more efficient approach would be to establish the reason why these SNNs do not scale with depth the same way as ANNs. We speculate that the fact that the signed version of our algorithm could circumvent the decrease of performance with the number of layers observed with both BPTT and burstprop is indicative of an issue with the scaling of the gradient. We hope that future work will establish means to circumvent this issue.

An important limitation of our results worth mentioning is that we limit our experimentation to MNIST-like datasets. MNIST itself is a fairly simple benchmark problem for supervised learning algorithms

and many non-neural network algorithms reach high test accuracy on this task. For instance, a linear classifier can achieve 7.6% error [66], a random forest algorithm can achieve 2.4% error, and an SVM can achieve 0.56% error [75]. The goal of this work was not to achieve state-of-the-art performance on this benchmark, but to show that Burstprop can be applied to solve a more complex learning task than XOR. Further experimentation with more complex problems is required to see how that algorithm scales.

Current neuromorphic hardware typically emulates leaky integrate-and-fire-like neurons, and rarely implements dendritic compartments [76, 77], bursting or burst-dependent plasticity. Our work suggests that developing such a device can allow spiking neural networks to learn on chip, which should improve substantially both the energy requirements and the speed of training.

Funding

This work was supported by a Vector Institute Scholarship to MS and a NSERC Discovery Grant (RGPIN-2024-05 162) to RN.

Ethical Statement

This work involved no human or animal subject and it was not a field study.

Data availability statement

The data cannot be made publicly available upon publication because they are not available in a format that is sufficiently accessible or reusable by other researchers. The data that support the findings of this study are available upon reasonable request from the authors.

ORCID iDs

Michael Stuck  <https://orcid.org/0009-0007-2064-7097>
 Xingyun Wang  <https://orcid.org/0000-0002-6192-4456>
 Richard Naud  <https://orcid.org/0000-0001-7383-3095>

References

- [1] Schuman C D, Kulkarni S R, Maryam Parsa J P M, Date P and Kay B 2022 Opportunities for neuromorphic computing algorithms and applications *Nat. Comput. Sci.* **2** 10–19
- [2] Zenke F, Bohté S M, Clopath C, Comşa I M, Göltz J, Maass W, Masquelier T, Naud R, Neftci E O and Petrovici M A *et al* 2021 Visualizing a joint future of neuroscience and neuromorphic engineering *Neuron* **109** 571–5
- [3] Yao M *et al* 2024 Spike-based dynamic computing with asynchronous sensing-computing neuromorphic chip *Nat. Commun.* **15** 4464
- [4] Roy K, Jaiswal A and Panda P 2019 Towards spike-based machine intelligence with neuromorphic computing *Nature* **575** 607–17
- [5] Gallego G *et al* 2020 Event-based vision: a survey *IEEE Trans. Pattern Anal. Mach. Intell.* **44** 154–80
- [6] Upadhyay N K, Jiang H, Wang Z, Asapu S, Xia Q and Joshua Yang J 2019 Emerging memory devices for neuromorphic computing *Adv. Mater. Technol.* **4** 1800589
- [7] Indiveri G and Liu S-C 2015 Memory and information processing in neuromorphic systems *Proc. IEEE* **103** 1379–97
- [8] Cramer B *et al* 2022 Surrogate gradients for analog neuromorphic computing *Proc. Natl Acad. Sci.* **119** e2109194119
- [9] Baldi P and Sadowski P 2016 A theory of local learning, the learning channel and the optimality of backpropagation *Neural Netw.* **83** 51–74
- [10] Payeur A, Guerguiev J, Zenke F, Richards B A and Naud R 2021 Burst-dependent synaptic plasticity can coordinate learning in hierarchical circuits *Nat. Neurosci.* **24** 1–10
- [11] Greedy W, Wei Zhu H, Pemberton J, Mellor J and Ponte Costa R 2022 Single-phase deep learning in cortico-cortical networks *Advances in Neural Information Processing Systems* p 35
- [12] Letzkus J J, Kampa B M and Stuart G J 2006 Learning rules for spike timing-dependent plasticity depend on dendritic synapse location *J. Neurosci.* **26** 10420–9
- [13] Bittner K C, Milstein A D, Grienberger C, Romani S and Magee J C 2017 Behavioral time scale synaptic plasticity underlies cal place fields *Science* **357** 1033–6
- [14] Caya-Bissonnette L, Naud R and Beique J-C 2023 Cellular substrate of eligibility traces during behavioral timescale synaptic plasticity *BioRxiv*
- [15] Friedenberger Z, Harkin E, Tóth K and Naud R 2023 Silences, spikes and bursts: three-part knot of the neural code *J. Physiol.* **601** 5165–93
- [16] Naud R and Sprekeler H 2018 Sparse bursts optimize information transmission in a multiplexed neural code *Proc. Natl Acad. Sci.* **115** e6329–38
- [17] Zhou R 2023 A method of converting ann to snn for image classification 2023 *IEEE 3rd Int. Conf. on Neuromorphic Computing (ICNC)* (IEEE)
- [18] Gao H, Junxian H, Wang H, Wang T, Zhong Z, Jianyi Y, Wang Y, Tian M and Shi C 2023 High-accuracy deep ANN-to-SNN conversion using quantization-aware training framework and calcium-gated bipolar leaky integrate and fire neuron *Front. Neurosci.* **17** 1141701

- [19] Hunsberger E and Eliasmith C 2016 Training spiking deep networks for neuromorphic hardware (arXiv:1611.05141)
- [20] Schmitt S et al 2017 Neuromorphic hardware in the loop: training a deep spiking network on the brainscales wafer-scale system *2017 International Joint Conference on Neural Networks (IJCNN)* (IEEE) pp 2227–34
- [21] Neftci E O, Mostafa H and Zenke F 2019 Surrogate gradient learning in spiking neural networks *IEEE Signal Processing Magazine* vol 36 (IEEE) pp 51–63
- [22] Bam Shrestha S and Orchard G 2018 Slayer: spike layer error reassignment in time *Advances in Neural Information Processing Systems*
- [23] Ellenberger B, Haider P, Jordan J, Max K, Jaras I, Kriener L, Benitez F and Petrovici M A 2024 Backpropagation through space, time, and the brain (arXiv:2403.16933)
- [24] Nessler David, Kappel B and Maass W 2014 STDP installs in winner-take-all circuits an online approximation to hidden markov model learning *PLoS Comput. Biol.* **10** e1003511
- [25] Diehl P U and Cook M 2015 Unsupervised learning of digit recognition using spike-timing-dependent plasticity *Front. Comput. Neurosci.* **9** 99
- [26] Legenstein R, Pecevski D and Maass W 2008 A learning theory for reward-modulated spike-timing-dependent plasticity with application to biofeedback *PLOS Comput. Biol.* **4** e1000180
- [27] Florian R V 2007 Reinforcement learning through modulation of spike-timing-dependent synaptic plasticity *Neural Comput.* **19** 1468–502
- [28] Frémaux N, Sprekeler H and Gerstner W 2010 Functional requirements for reward-modulated spike-timing-dependent plasticity *J. Neurosci.* **30** 13326–37
- [29] Körding K P and König P 2001 Supervised and unsupervised learning with two sites of synaptic integration *J. Comput. Neurosci.* **11** 207–15
- [30] Sacramento J, ao, Ponte Costa R, Bengio Y and Senn W 2018 Dendritic cortical microcircuits approximate the backpropagation algorithm *Advances in Neural Information Processing Systems* pp 8721–32
- [31] Guerguiev J, Lillicrap T P and Richards B A 2017 Towards deep learning with segregated dendrites *eLife* **6** e22901
- [32] Reyes A, Lujan R, Rozov A, Burnashev N, Somogyi P and Sakmann B 1998 Target-cell-specific facilitation and depression in neocortical circuits *Nat. Neurosci.* **1** 279–85
- [33] Scanziani M, Gähwiler B H and Charpak S 1998 Target cell-specific modulation of transmitter release at terminals from a single axon *Proc. Natl Acad. Sci.* **95** 12004–9
- [34] Beninger J, Rossbroich J, Toth K and Naud R 2024 Functional subtypes of synaptic dynamics in mouse and human *Cell Rep.* **43** 113785
- [35] Larkum M E, Zhu J and Sakmann B 1999 A new cellular mechanism for coupling inputs arriving at different cortical layers *Nature* **398** 338–41
- [36] Friedenberger Z and Naud R 2023 Dendritic excitability primarily controls overdispersion *Nat. Comput. Sci.* **4** 19–28
- [37] Stuck M and Naud R 2023 Burstprop for learning in spiking neuromorphic hardware *Proc. 2023 Int. Conf. on Neuromorphic Systems (ICONS)* pp 1–5
- [38] Lillicrap T P, Cownden D, Tweed D B and Akerman C J 2016 Random synaptic feedback weights support error backpropagation for deep learning *Nat. Commun.* **7** 13276
- [39] Nøkland A 2016 Direct feedback alignment provides learning in deep neural networks *Advances in neural information processing systems* p 29
- [40] Samadi A, Lillicrap T P and Tweed D B 2017 Deep learning with dynamic spiking neurons and fixed feedback weights *Neural Comput.* **29** 578–602
- [41] Akrouf M, Wilson C, Humphreys P, Lillicrap T and Tweed D B 2019 Deep learning without weight transport *Advances in Neural Information Processing Systems* p 32
- [42] Ji-An Li and Benna M K 2024 Deep learning without weight symmetry (arXiv:2405.20594)
- [43] Max K, Kriener L, Pineda García G, Nowotny T, Jaras I, Senn W and Petrovici M A 2024 Learning efficient backprojections across cortical hierarchies in real time *Nat. Mach. Intell.* **6** 619–30
- [44] Podlaski B and Machens C K 2020 Biological credit assignment through dynamic inversion of feedforward networks *Advances in Neural Information Processing Systems* vol 33 pp 10065–76
- [45] Bartunov S, Santoro A, Richards B, Marris L, Hinton G E and Lillicrap T 2018 Assessing the scalability of biologically-motivated deep learning algorithms and architectures *Advances in Neural Information Processing Systems* p 31
- [46] Moskowitz T H, Litwin-Kumar A and Abbott L F 2018 Feedback alignment in deep convolutional networks (arXiv:1812.06488)
- [47] Jiménez Sanfiz A and Akrouf M 2021 Benchmarking the accuracy and robustness of feedback alignment algorithms (arXiv:2108.13446)
- [48] Zenke F and Ganguli S 2018 Superspike: supervised learning in multilayer spiking neural networks *Neural Comput.* **30** 1514–41
- [49] Zhao D, Zeng Y, Zhang T, Shi M and Zhao F 2020 GLSNN: a multi-layer spiking neural network based on global feedback alignment and local stdp plasticity *Front. Comput. Neurosci.* **14** 576841
- [50] Xing X, Zhang Z, Ziyi N, Xiao S, Yiming J, Fan S, Wang Y, Zhang J and Guoqi Li 2024 SpikeLM: towards general spike-driven language modeling via elastic bi-spiking mechanisms (arXiv:2406.03287)
- [51] Kolen J F and Pollack J B 1994 Backpropagation without weight transport *Proc. 1994 IEEE Int. Conf. on Neural Networks (ICNN)* (IEEE) pp 1375–80
- [52] Kendra S B 2015 Mirrored STDP implements autoencoder learning in a network of spiking neurons *PLOS Comput. Biol.* **11** e1004566
- [53] Naud R and Gerstner W 2012 The performance (and limits) of simple neuron models: generalizations of the leaky integrate-and-fire model computational systems neurobiology. *Computational Systems Neurobiology* ed L N Nicolas (Springer)
- [54] Naud R, Wang X, Friedenberger Z, Payeur A, Shin J N, Béique J-C, Richards B A, Drüke M, Larkum M E and Doron G 2024 Fast burst fraction transients convey information independent of the firing rate *eLife* **13** R98290
- [55] Gerstner W, Kistler W, Naud R and Paninski L 2014 *Neuronal Dynamics* (Cambridge University Press)
- [56] Mensi S, Naud R, Avermann M, Petersen C C H and Gerstner W 2012 Parameter extraction and classification of three neuron types reveals two different adaptation mechanisms *J. Neurophys.* **107** 1756–75
- [57] Gerstner W and Naud R 2009 How good are neuron models? *Science* **326** 379–80
- [58] Pozzorini C, Naud R, Mensi S and Gerstner W 2013 Temporal whitening by power-law adaptation in neocortical neurons *Nat. Neurosci.* **16** 942–8

- [59] Naud R, Bathellier B and Gerstner W 2014 Spike-timing prediction in cortical neurons with active dendrites *Front. Comput. Neurosci.* **8** 90
- [60] Harkin E F, Lynn M B, Payeur A, Boucher J-F, Caya-Bissonnette L, Cyr D, Stewart C, Longtin A, Naud R and Béique J-C 2023 Temporal derivative computation in the dorsal raphe network revealed by an experimentally-driven augmented integrate-and-fire modeling framework *eLife* **12** e72951
- [61] Payeur A, Béique J-C and Naud R 2019 Classes of dendritic information processing *Curr. Opinion Neurobiol.* **58** 78–85
- [62] Xu N-long, Harnett M T, Williams S R, Huber D, Connor D H O, Svoboda K and Magee J C 2012 Nonlinear dendritic integration of sensory and motor input during an active sensing task *Nature* **492** 247–51
- [63] Harkin E F, Shen P R, Goel A, Richards B A and Naud R 2022 Parallel and recurrent cascade models as a unifying force for understanding subcellular computation *Neuroscience* **489** 200–15
- [64] Williams E, Payeur A, Gidon A and Naud R 2021 Neural burst codes disguised as rate codes *Sci. Rep.* **11** 15910
- [65] Rossbroich J, Gyga J and Zenke F 2022 Fluctuation-driven initialization for spiking neural network training *Neuromorph. Comput. Eng.* **2** 044016
- [66] LeCun Y, Bottou L, Bengio Y and Haffner P 1998 Gradient-based learning applied to document recognition *Proc. IEEE* **86** 2278–324
- [67] Senn W, Dold D, Kungl A F, Ellenberger B, Jordan J, Bengio Y, Sacramento J ao and Petrovici M A 2024 A neuronal least-action principle for real-time learning in cortical circuits *eLife* **12** RP89674
- [68] Cohen G, Afshar S, Tapson J and Van Schaik A 2017 EMNIST: extending MNIST to handwritten letters 2017 *Int. Joint Conf. on Neural Networks (IJCNN)* (IEEE) pp 2921–6
- [69] Xiao H, Rasul K and Vollgraf R 2017 Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms (arXiv:1708.07747)
- [70] Dipu Kabir H M, Abdar M, Khosravi A, Mohammad Jafar Jalali S, Atiya A F, Nahavandi S and Srinivasan D 2022 SpinalNet: deep neural network with gradual input *IEEE Trans. Artif. Intell.* **4** 1165–77
- [71] Taylor L, King A and Harper N 2022 Robust and accelerated single-spike spiking neural network training with applicability to challenging temporal tasks (arXiv:2205.15286)
- [72] Kemker R, McClure M, Abitino A, Hayes T and Kanan C 2018 Measuring catastrophic forgetting in neural networks *Proc. AAAI Conf. on Artificial Intelligence* vol 32
- [73] Jiang X, Shen S, Cadwell C R, Berens P, Sinz F, Ecker A S, Patel S and Tolias A S 2015 Principles of connectivity among morphologically defined cell types in adult neocortex *Science* **350** aac9462
- [74] Lorenz K M and Indiveri G 2015 Rounding methods for neural networks with low resolution synaptic weights (arXiv:1504.05767)
- [75] Liu Y 2021 *The Image Classification of Mnist Dataset by Using Machine Learning Techniques* (UC Merced Previously Published Works)
- [76] Kaiser J, Billaudelle S, Müller E, Tetzlaff C, Schemmel J and Schmitt S 2022 Emulating dendritic computing paradigms on analog neuromorphic hardware *Neuroscience* **489** 290–300
- [77] Baek E, Song S, Baek C-K, Rong Z, Shi L and Vittorio Cannistraci C 2024 Neuromorphic dendritic network computation with silent synapses for visual motion perception *Nat. Electron.* **7** 1–12